

Los errores que convierten tu aplicación Blazor en una aplicación lenta

Y cómo arreglarlos midiendo cada cambio

Emiliano Montesdeoca del Puerto

NetCoreConf Madrid 2026 · 23 de octubre

Sponsors



‹NetCoreConf/›
Madrid '26

Platinum sponsors

Inkoova

intelequia

NTT DATA

TOKIOTA

V-Valley

Gold sponsors

axazure by avvale

plain concepts

LIDL

SCRM INTERNATIONAL HUB

Colabora

sessionize

Hola, soy Emi

- No he estado nunca en la cárcel
- Papá y felizmente casado
- Cloud Solutions Team Lead · Intelequia
- Microsoft MVP en Developer Technologies desde 2022
- Blogger / intento de influencer





¿Qué es Blazor?

Un framework de **ASP.NET Core** para crear interfaces web con **componentes Razor: HTML y C#**.

```
<button @onclick="Incrementar">Clics: @clics</button>

@code {
    private int clics;
    private void Incrementar() => clics++;
}
```

Cada componente tiene su interfaz, su estado y sus eventos.



¿Dónde se ejecuta una aplicación Blazor?

- **Static SSR:** el servidor genera HTML por petición.
- **Interactive Server:** C# y el renderizado, en el servidor.
  Los eventos y las actualizaciones del DOM viajan por SignalR.
- **Interactive WebAssembly:** C# y el renderizado, en el navegador.

Un componente puede renderizarse sin que cambie el DOM.



La aplicación que vamos a usar

BlazorLento: una aplicación de pedidos con un buscador y un carrito.

1. Las filas **se renderizan aunque sus datos no cambien**.
2. El buscador llama a la API **con cada tecla**.
3. La tabla carga **miles de filas que no se ven**.
4. Varios componentes **repiten la misma consulta**.
5. Añadir al carrito **hace que la tabla se vuelva a renderizar**.



ANTES / 1. Las filas se renderizan sin haber cambiado

```
@code {
    [Parameter, EditorRequired] public Pedido Pedido { get; set; } = default!;
    private int _renders;
}
```

El padre se renderiza cada segundo y pasa el mismo `Pedido`, una **clase mutable**.
Esta fila no evita renderizarse aunque los datos que muestra sigan igual.

Cada segundo se renderizan las 200 filas, aunque sus datos sigan igual.



DESPUÉS / La fila se actualiza solo si cambia el pedido

200 renders/s → 0 · pedido modificado → 1

```
private int _idRenderizado = -1;
private int _versionRenderizada = -1;

protected override bool ShouldRender()
    => Pedido.Id != _idRenderizado || Pedido.Version != _versionRenderizada;

protected override void OnAfterRender(bool firstRender)
{
    _idRenderizado = Pedido.Id;
    _versionRenderizada = Pedido.Version;
}
```



Qué garantiza esta transformación

- Guarda `Id` y `Version` tras el primer renderizado; por eso la carga inicial se conserva.
- Incrementa `Version` cuando cambia un dato visible de la fila.
- También puedes pasar **parámetros primitivos** que Blazor compare.

No usar `ShouldRender => false` como receta general.



PARA TU IA / Copia este prompt

> Revisa la fila de esta app Blazor: recibe un `Pedido` mutable y su padre se actualiza periódicamente. Evita renderizar las filas cuyos datos visibles no cambian, pero conserva la actualización cuando sí cambian. No uses `ShouldRender => false` como atajo. Inspecciona cómo se modifican los pedidos, implementa la opción más sencilla y verifica ambos casos.



ANTES / 2. Una llamada a la API por cada tecla

```
private async Task Buscar(ChangeEventArgs e)
{
    _texto = e.Value?.ToString() ?? "";
    var texto = _texto;
    _resultados = await Api.BuscarAsync(texto);
    _textoResultados = texto;
}
```

`@oninput` + **martinez** = **8 llamadas**.

Una búsqueda anterior puede terminar después y sustituir los resultados actuales.



DESPUÉS / La búsqueda espera y cancela la anterior

8 llamadas → 1 · resultados actuales

```
var texto = _texto;
_cts?.Cancel();
_cts?.Dispose();
_cts = new CancellationTokenSource();
var ct = _cts.Token;

try
{
    await Task.Delay(DebounceMs, ct);
    var resultados = await Api.BuscarAsync(texto, ct);
    ct.ThrowIfCancellationRequested();
    _resultados = resultados;
    _textoResultados = texto;
}
catch (OperationCanceledException) when (ct.IsCancellationRequested) { }
```



Debounce y cancelación

- **Debounce:** espera 300 ms desde la última tecla antes de buscar.
- **Cancelación:** detiene la espera o la llamada anterior cuando cambia el texto.
- **Protección:** no muestra una respuesta si ya hay una búsqueda más reciente.
- Al salir de la página, `Dispose` cancela la operación pendiente.



PARA TU IA / Copia este prompt

> Analiza el buscador de esta app Blazor, que llama a la API en cada `oninput`. Añade debounce y cancelación de la búsqueda anterior; pasa el token a la API y evita que una respuesta antigua reemplace los resultados actuales. Cancela y libera el token al salir del componente, conserva la experiencia existente y verifica la escritura rápida, la pausa y la cancelación. No ocultes errores distintos de una cancelación esperada.



ANTES / 3. Cargar 5.000 filas para mostrar unas pocas

```
@foreach (var p in _pedidos)
{
    <tr data-pedido="@p.Id">
        <td>@p.Id</td>
        <td>@p.Cliente</td>
        <td>@p.Producto</td>
        <td class="num">@p.Cantidad</td>
        <td class="num">@p.Importe</td>
        <td>@p.Estado</td>
    </tr>
}
```

5.000 filas × 6 celdas = 30.000 celdas.

El contenedor solo muestra unas **15 filas** a la vez.



DESPUÉS / La transformación: virtualizar la tabla

```
<Virtualize TItem="Pedido" Items="_pedidos" ItemSize="38"
            OverscanCount="5" SpacerElement="tr" Context="p">
    <tr data-pedido="@p.Id">
        <td>@p.Id</td>
        <td>@p.Cliente</td>
        <td>@p.Producto</td>
        <td class="num">@p.Cantidad</td>
        <td class="num">@p.Importe.ToString("C", CultureInfo.GetCultureInfo("es-ES"))</td>
        <td>@p.Estado</td>
    </tr>
</Virtualize>
```

5.000 filas → visibles + margen



Virtualizar no es paginar los datos

Implementación	Filas en el DOM	Datos solicitados
`foreach`	5.000	5.000
`Virtualize` + `Items`	Visibles + margen	5.000
`Virtualize` + `ItemsProvider`	Visibles + margen	Solo los necesarios

Renderizar menos filas y **cargar menos datos** son mejoras distintas.



PARA TU IA / Copia este prompt

> Revisa esta lista Blazor que genera miles de filas con `foreach`. Sustitúyela por `Virtualize` y conserva el marcado válido dentro de la tabla. Usa `Items` si los datos ya están cargados; si la lista debe ser grande también en memoria, propone `ItemsProvider` con paginación y cancelación. Comprueba la altura real de las filas, el número visible en el DOM y que se puede recorrer hasta el final. No confundas virtualización con paginación.



ANTES / 4. Volver a cargar el mismo pedido

```
<DetallePedidoLento PedidoId="_pedidoId">
    Vista actualizada a las @_hora.ToString("HH:mm:ss")
</DetallePedidoLento>
```

El reloj cambia cada segundo, pero el **Id del pedido sigue igual**.

Al pasar el reloj dentro del detalle, se repite la consulta.



DESPUÉS / El reloj fuera; el Id dentro

mismo Id → 0 llamadas · Id nuevo → 1

```
<p class="small muted">
    Vista actualizada a las @_hora.ToString("HH:mm:ss")
</p>
<DetallePedidoRapido PedidoId="_pedidoId" />
```



PARA TU IA / Copia este prompt

> Revisa esta página Blazor: un texto del reloj cambia cada segundo dentro de un componente que carga un pedido. Separa ese texto del detalle para que el componente reciba solo el Id. Verifica que los cambios del reloj no vuelvan a llamar a la API y que cambiar el Id sí cargue el pedido nuevo. Haz el cambio mínimo y conserva la cancelación si se cambia de Id durante una carga.



ANTES / 5. La tabla se renderiza al cambiar el carrito

```
public event Action? OnChange;

public void AñadirAlCarrito(int pedidoId)
{
    Carrito.Add(pedidoId);
    OnChange?.Invoke();
}
```

Un evento general avisa a todos cuando cambia cualquier dato.

Al añadir un producto, la tabla se renderiza aunque sus pedidos no cambien.



DESPUÉS / Notificar solo el cambio que necesita cada componente

Añadir: tabla 1 render → 0

```
public event Action? CarritoCambiado;
public event Action? FiltroCambiado;

CarritoCambiado?.Invoke(); // Al añadir al carrito
FiltroCambiado?.Invoke(); // Al cambiar el filtro
```

La tabla solo escucha a `FiltroCambiado`.



PARA TU IA / Copia este prompt

> Al añadir un producto al carrito, la tabla de pedidos también se renderiza, aunque no usa el carrito. Cambia lo mínimo para que carrito y filtro tengan notificaciones separadas y la tabla escuche solo al filtro. Conserva la desuscripción en `Dispose` y comprueba que Añadir no actualiza la tabla, pero cambiar el filtro sí.



Cinco preguntas para revisar tu aplicación

Pregunta	Qué revisar	Qué medir
¿Se renderiza sin cambios?	Parámetros y `ShouldRender`	Renderizados
¿Busco con cada tecla?	Debounce y cancelación	Llamadas
¿Renderizo filas que no se ven?	`Virtualize` y paginación	Filas y lotes
¿Vuelve a cambiar el Id?	Parámetros estables	Llamadas repetidas
¿Qué componentes deben actualizarse?	Suscripciones de estado	Componentes afectados

No hace falta optimizarlo todo, sino corregir lo que está causando la lentitud.



ONE MORE TIP / 1 · Agrupa llamadas relacionadas

Idea: si la pantalla siempre necesita varios datos juntos, pídelos a un único endpoint que devuelva un DTO.

- Reduce viajes de red, no solo tiempo de espera.
- `Task.WhenAll` paraleliza, pero no reduce el número de solicitudes.

```
var panel = await http.GetFromJsonAsync<PanelDto>(
    $"/api/pedidos/{id}/panel", ct);
```



ONE MORE TIP / 2 · Cachea lecturas que cambian poco

Idea: evita volver a leer datos que siguen vigentes.

- Define una duración y cuándo invalidar.
- No compartas datos de usuario entre sesiones.

```
if (cache.TryGetValue("productos",
        out IReadOnlyList<Producto>? productos)
    && productos is not null)
    return productos;

productos = await api.GetProductosAsync(ct);
cache.Set("productos", productos, TimeSpan.FromMinutes(5));
return productos;
```



ONE MORE TIP / 3 · Trae solo los campos que muestras

Idea: para una pantalla de lectura, proyecta los campos necesarios.

- `AsNoTracking` evita seguimiento si solo vas a leer.
- Menos columnas y objetos reducen memoria y tráfico.

```
var filas = await db.Pedidos
    .AsNoTracking()
    .Where(p => p.Estado == estado)
    .Select(p => new PedidoFila(p.Id, p.Cliente, p.Importe))
    .ToListAsync(ct);
```



ONE MORE TIP / 4 · Pagina desde el servidor

Idea: en colecciones grandes, solicita una página y pide la siguiente al desplazarte.

- Usa un orden estable y limita el tamaño de página.
- Para desplazamientos muy profundos, valora paginación por cursor.

```
var pagina = await db.Pedidos
    .OrderBy(p => p.Id)
    .Skip(offset)
    .Take(pageSize)
    .Select(p => new PedidoFila(p.Id, p.Cliente, p.Importe))
    .ToListAsync(ct);
```



ONE MORE TIP / 5 · Mide antes y después

Idea: compara la misma operación con los mismos datos y condiciones.

- Mide lo que le importa al usuario y registra el volumen de datos.
- No conviertas una medición aislada en una promesa universal.

```
var inicio = Stopwatch.GetTimestamp();
var pedidos = await api.GetPedidosAsync(200, ct);
logger.LogInformation(
    "Pedidos={Count}; {ElapsedMs} ms",
    pedidos.Count,
    Stopwatch.GetElapsedTime(inicio).TotalMilliseconds);
```

Sponsors



‹NetCoreConf/›
Madrid '26

Platinum sponsors

Inkoova

intelequia

NTT DATA

TOKIOTA

V-Valley

Gold sponsors

axazure by avvale

plain concepts

LIDL

SCRM INTERNATIONAL HUB

Colabora

sessionize



Gracias

¿Preguntas?

Emiliano Montesdeoca
linkedin.com/in/emimontesdeoca · @emimontesdeocaa